Road Anomaly Detection Using YOLOv8

Course: Image and Video Processing

Mini-Project Report

Submitted by

Abhinav Prabhat (H009)

Atharva Srivastava (H012)

Diya Saxena (H021)

Programme / Semester / Division

MBATech CE / SEM V / A

Faculty Name: Dr. RajGaurav Mishra

Institute / Department: STME, NMIMS INDORE

Academic Year: 2026-27

Abstract

Road anomalies such as cracks, potholes, and speed bumps pose serious safety risks and increase vehicle maintenance costs. Manual inspection is slow, subjective, and difficult to scale for large road networks. This project presents an automated Road Anomaly Detection system built with deep- learning- based object detection. We trained a custom YOLOv8m model on a unified dataset of 30,685 images compiled from multiple road- damage and traffic datasets, covering seven classes: Heavy-Vehicle, Light-Vehicle, Pedestrian, Crack, Crack-Severe, Pothole, and Speed- Bump. The pipeline includes data preprocessing, label standardization, class balancing through augmentation, and class- weighted training. The best model achieved an overall precision of 0.736, recall of 0.740, mAP@0.5 of 0.745, and F1-score of 0.738 on the test set, with inference speeds around 12 ms per image. A Streamlit-based web application was also developed to perform inference on images, videos, and live camera feeds. The system demonstrates that modern object-detection architectures can effectively assist in road-condition monitoring and traffic-scene understanding.

1. Introduction

1.1 Background of the Problem

Road infrastructure is one of the most critical assets of any country. Poor road conditions not only affect ride quality and fuel efficiency but also contribute significantly to accidents and vehicle damage. Traditional methods of road inspection rely on manual surveys, which are labor-intensive, time-consuming, and prone to human error. With the rapid growth of urban areas and highway networks, there is an urgent need for automated systems that can continuously monitor road conditions.

1.2 Relevance of the Problem

Automated road anomaly detection is relevant to multiple stakeholders, including municipal corporations, highway authorities, logistics companies, and autonomous vehicle developers. Early detection of potholes and cracks enables timely repairs, reduces long-term maintenance costs, and improves road safety. In the context of Advanced Driver Assistance Systems (ADAS), real-time detection of speed bumps, pedestrians, and vehicles further supports safer driving.

1.3 Role of Image and Video Processing

Image and video processing techniques form the foundation of automated road inspection. Tasks such as image acquisition, preprocessing, feature extraction, object detection, and video-frame analysis allow computers to interpret visual road data. Modern deep-learning models, particularly You Only Look Once (YOLO) architectures, have made it possible to detect multiple objects in real time with high accuracy, making them suitable for both offline analysis and live deployment.

1.4 Motivation for Selecting the Project

The motivation for this project stems from the visible problem of deteriorating road conditions and the potential of computer vision to address it. By combining road-damage detection with traffic-object detection, the project offers a practical, real-world application of image and video processing concepts. The availability of open datasets and accessible deep-learning frameworks such as Ultralytics YOLOv8 made the implementation feasible within the scope of a mini-project.

2. Problem Statement and Objectives

2.1 Real-World Problem

The project addresses the problem of automatically identifying road surface anomalies and relevant traffic objects from images and videos. Given an input image or video frame captured from a road scene, the system must detect and classify objects such as cracks, severe cracks, potholes, speed bumps, heavy vehicles, light vehicles, and pedestrians.

2.2 Objectives

- To collect, merge, and standardize multiple road-damage and traffic datasets into a unified YOLO-compatible dataset.
- To train a custom YOLOv8m object-detection model capable of detecting seven road-related classes.
- To evaluate model performance using standard metrics such as precision, recall, mAP@0.5, mAP@0.5:0.95, and F1-score.
- To compare the custom model against a pre-trained YOLOv8s model trained on the CRDDC2022 dataset.
- To build a user-friendly Streamlit web application for image, video, and live-camera inference.

3. Scope of the Project

3.1 What the Project Covers

- Detection of road anomalies (cracks, severe cracks, potholes) and traffic objects (heavy vehicles, light vehicles, pedestrians, speed bumps).
- Training and evaluation of a custom YOLOv8m model on a curated dataset of approximately 30,685 images.
- Data augmentation and class weighting to handle dataset imbalance.
- Inference on static images, pre-recorded videos, and live camera streams.
- A web-based demo application built with Streamlit.

3.2 What the Project Does Not Cover

- Depth estimation or 3D reconstruction of road anomalies.
- Automatic routing of repair crews or integration with government maintenance portals.
- Detection of lane markings, traffic signs, or traffic signals.
- Deployment on edge devices such as Raspberry Pi or mobile phones.

3.3 Input Types and Application Domain

The system accepts static RGB images and video files. The application domain includes smart-city road monitoring, ADAS development, fleet- management dash- cam analytics, and civil infrastructure inspection.

4. Dataset / Input Description

4.1 Data Sources

The final dataset was created by merging and standardizing four publicly available sources:

- RAD Dataset: approximately 8,400 images.
- Indian Roads Dataset: approximately 5,100 images.
- Humps/Bumps/Potholes Dataset: approximately 3,200 images.
- HighRPD Dataset: approximately 11,700 images, preprocessed from XML annotations to YOLO format.

4.2 Dataset Composition

After preprocessing, label standardization, merging, and augmentation, the final unified dataset contains 30,685 RGB images split into training (23,321), validation (4,518), and test (2,846) sets. All annotations follow the YOLO format with normalized bounding-box coordinates.

4.3 Unified Classes

Class	Description
Heavy-Vehicle	Large vehicles such as trucks and buses
Light-Vehicle	Cars, motorcycles, and smaller vehicles
Pedestrian	People walking on or near the road
Crack	Surface cracks of any orientation
Crack-Severe	Wider or more extensive cracks
Pothole	Depressions or holes in the road surface
Speed-Bump	Raised road structures for speed control

4.4 Preprocessing and Augmentation

HighRPD XML annotations were converted to YOLO format and class labels were mapped. All datasets were standardized to seven unified classes. Prefixes were added to image filenames to maintain traceability, and image-label pairs were verified. The training set was analyzed for class imbalance; minority classes (Heavy-Vehicle, Pedestrian, Speed-Bump) were augmented using flips, brightness adjustments, and rotations, adding 5,316 training images. Final class weights were computed for use during training.

5. Methodology / Proposed Approach

The complete processing pipeline consists of the following stages:

Stage 1: Data Collection

Images and annotations were gathered from the RAD, Indian Roads, Humps/ Bumps/ Potholes, and HighRPD datasets.

Stage 2: Preprocessing

HighRPD annotations were converted from XML to YOLO format, classes were mapped, and the dataset was split into 70% train, 20% validation, and 10% test sets.

Stage 3: Label Standardization

All datasets were standardized to a common set of seven classes with consistent class indices.

Stage 4: Merging and Splitting

Datasets were merged, prefixes were added to filenames, and image-label pairs were verified. The initial merged dataset contained 25,369 images.

Stage 5: Balancing via Augmentation

Training-set class distribution was analyzed. Minority classes were augmented through geometric and photometric transformations, yielding 5,316 additional training images.

Stage 6: Model Training

A YOLOv8m model was trained for 120 epochs on an NVIDIA GeForce RTX 3060 Laptop GPU with class weights to mitigate residual imbalance.

Stage 7: Evaluation and Deployment

The trained model was evaluated on the test set and integrated into a Streamlit web application for image, video, and live-camera inference.

6. Techniques / Algorithms Used

6.1 Image Processing Techniques

- Resizing and normalization of input images to the 640x640 resolution required by YOLOv8.
- Color-space handling: RGB images are used directly by the YOLOv8 backbone.
- Data augmentation: horizontal and vertical flips, brightness adjustments, and rotations to increase training diversity.
- Bounding- box annotation conversion: XML- to- YOLO format mapping and class- label unification.

6.2 Deep-Learning Algorithms

- YOLOv8m (custom trained): primary detection model for the seven-class task.
- YOLOv8s (pre- trained): secondary model trained on CRDDC2022 for crack and pothole detection, used for comparison and optional fusion.
- Class- weighted loss: minority classes receive higher weights during training to counter imbalance.

6.3 Video Processing Techniques

- Frame extraction from uploaded video files using OpenCV.
- Per-frame object detection with YOLOv8 inference.
- Annotation overlay: bounding boxes and class labels drawn on each frame.
- Reconstruction of annotated frames into an output video stream.

7. Implementation

7.1 Programming Language and Tools

The project was implemented in Python 3.11. The primary tools and libraries used were OpenCV for image/video I/O, NumPy for numerical operations, Matplotlib for visualization, PyTorch as the deep-learning backend, and the Ultralytics YOLOv8 framework for model training and inference. The web demo was built using Streamlit.

7.2 Hardware

Model training was performed on an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB of VRAM. The custom YOLOv8m model was trained for 120 epochs and required approximately 27.8 hours.

7.3 Important Implementation Steps

1. Collected raw datasets and converted HighRPD XML annotations to YOLO format.
2. Standardized all labels to seven unified classes and verified image-label pairs.
3. Merged datasets and performed an initial 70/20/10 split.
4. Analyzed training-set class imbalance and augmented minority classes.
5. Computed class weights and trained YOLOv8m for 120 epochs.
6. Evaluated the best checkpoint on the held-out test set.
7. Developed Streamlit and partial Flask interfaces for interactive inference.

7.4 Key Files

The repository contains train.ipynb for model training, main.py and run.py for single- model inference, run2model.py for dual-model inference, and visualize_annotations_data.py for dataset exploration.

8. Results and Discussion

8.1 Validation Performance

The best YOLOv8m checkpoint was selected based on validation metrics. Table 1 summarizes the per-class validation performance.

Class	Precision	Recall	mAP@.5	mAP@.5:.95
Overall	0.738	0.726	0.733	0.443
Heavy-Vehicle	0.921	0.976	0.979	0.764
Light-Vehicle	0.894	0.965	0.967	0.659
Pedestrian	0.838	0.903	0.910	0.494
Crack	0.553	0.430	0.454	0.219
Crack-Severe	0.526	0.467	0.471	0.265
Pothole	0.595	0.432	0.432	0.171
Speed-Bump	0.842	0.911	0.919	0.530

Table 1: Validation performance of the custom YOLOv8m model.

8.2 Test Set Performance

A final evaluation was conducted on the dedicated test set. Table 2 presents the per-class test results.

Class	Precision	Recall	mAP@.5	mAP@.5:.95
Overall	0.736	0.740	0.745	0.448
Heavy-Vehicle	0.913	0.978	0.981	0.763
Light-Vehicle	0.892	0.951	0.961	0.649
Pedestrian	0.822	0.915	0.918	0.522
Crack	0.576	0.484	0.505	0.240
Crack-Severe	0.548	0.503	0.493	0.273
Pothole	0.597	0.440	0.468	0.198
Speed-Bump	0.804	0.908	0.885	0.487

Table 2: Test-set performance of the custom YOLOv8m model.

8.3 Overall Metrics Summary

- Precision: 0.736
- Recall: 0.740
- mAP@0.5: 0.745
- mAP@0.5:0.95: 0.448

- F1-Score: 0.738
- Average inference speed: ~12.0 ms per image

8.4 Discussion

The model performs strongly on large, well-defined classes such as Heavy-Vehicle, Light-Vehicle, and Speed-Bump, achieving mAP@0.5 values above 0.88. Pedestrian detection is also reliable. Performance is lower for Crack, Crack-Severe, and Pothole classes because these objects are often small, low-contrast, and visually similar to road texture. The gap between mAP@0.5 and mAP@0.5:0.95 indicates that while the model usually finds the objects, precise bounding-box localization remains challenging for the damage classes.

A sample processed video demonstrating real-time detection is available, and evaluation visualizations including the confusion matrix, precision-recall curve, F1 curve, precision curve, and recall curve were generated during testing.

9. Challenges and Limitations

- Class imbalance: cracks and potholes were under- represented relative to vehicles. Augmentation and class weights partially mitigated this issue.
- Small object detection: distant or small cracks and potholes are difficult to localize accurately.
- Inter- class similarity: Crack and Crack- Severe share visual features, leading to occasional misclassification.
- Environmental variability: lighting changes, shadows, wet roads, and occlusions affect detection reliability.
- Hardware constraints: training was limited to a 6 GB laptop GPU, requiring a batch size of 9 and longer training time.
- Flask interface is still under construction and not yet fully functional.

10. Conclusion

This project successfully implemented an automated Road Anomaly Detection system using the YOLOv8 object-detection architecture. A custom YOLOv8m model was trained on a unified dataset of 30,685 images spanning seven classes. The system achieved an overall test precision of 0.736, recall of 0.740, mAP@0.5 of 0.745, and F1-score of 0.738, with inference speeds near 12 ms per image.

Key techniques employed include dataset merging, label standardization, data augmentation for class balancing, class-weighted training, and frame-by-frame video inference. A Streamlit web application was developed to make the model accessible for image, video, and live-camera inputs.

Through this project, we gained practical experience in end-to-end image and video processing workflows, from data preparation and model training to evaluation and deployment. We also learned the importance of handling class imbalance and the challenges of detecting small, low-contrast road anomalies in real-world conditions.

References

- Jocher, G., et al. (2023). Ultralytics YOLOv8. Available at: https:// github.com/ ultralytics/ ultralytics
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Arya, D., et al. (2021). Deep Learning-Based Road Damage Detection and Classification for Multiple Countries. Transportation Research Record.
- HighRPD Dataset contributors. High-Resolution Pothole Dataset.
- CRDDC2022: Global Road Damage Detection Challenge 2022 dataset.
- Project repository: https://github.com/collabdoor/Road-Anomaly-Detection
- Secondary model source: https://github.com/oracl4/RoadDamageDetection